What Happens on a Network?

A Custom Packet Sniffer in Python

1. Introduction

When we browse the web or watch a video, we rarely think about the packets carrying those bytes across the network. Firewalls, routers, and switches are constantly forwarding tiny chunks of data, but most of the time we only see the final result: a web page, an API response, or a streaming video.

This project explores the packet level view of networking by building a custom network packet sniffer in Python. The sniffer runs on Linux and macOS, captures live traffic from a chosen network interface, decodes key protocol headers (Ethernet, IPv4, ICMP, TCP, UDP), and exports captured data in .pcap format for analysis in Wireshark.

Instead of treating networking as an abstract diagram of "layers," this project focuses on the concrete question:

What actually happens on the wire when data moves across the network?

By writing my own sniffer, I forced myself to understand not only the theory behind protocols but also how they look as raw bytes, how tools capture them, and how to design a usable command line tool for analysis.

2. Aim of the Project

The project has four main aims:

1. Understand how data travels across networks

   I wanted to see real packets generated by my system, not just static examples in slides. Capturing live traffic provides a realistic view of how protocols behave in everyday use.

2. Learn how packets are structured and decoded

   Rather than relying entirely on Wireshark or high level libraries, I wanted to manually decode headers like Ethernet, IPv4, TCP, UDP, and ICMP in Python. This forces a deeper understanding of fields and bit layouts.

3. Build a practical, usable analysis tool

   The goal was not only to parse packets but to present them in a way that a human analyst can quickly read. That means structured output, colors, payload previews, and the ability to save data for later analysis.

4. Strengthen security and problem solving skills

Getting a sniffer working across Linux and macOS meant dealing with real world issues: permissions, OS differences, signal handling, and debugging. This project is a concrete demonstration of my ability to design, implement, debug, and document a security focused tool end to end.

3. Motivation and Context

There are many existing tools for packet analysis, including tcpdump and Wireshark. However, using them alone does not guarantee a deep understanding of what they are doing internally. By building my own sniffer, I gained:

- A more detailed mental model of how packets move from the wire into user space.
- Confidence in working with raw bytes, network headers, and binary formats.
- Practical experience with Python as a systems and security language, not just for scripts or data analysis.

From a career perspective, this project also serves as:

- A portfolio piece showing I can work at packet level, not just at application or log level.
- Evidence of problem solving: handling Scapy callbacks, Ctrl+C behavior, pcap writing, and cross platform issues.
- A foundation for more advanced security tools such as intrusion detection, traffic classification, or anomaly detection.

4. Tools and Technologies

The project deliberately uses a small set of focused tools:

- Python
  - Core language for implementing the sniffer.
  - Handles command line parsing, control flow, and some low level operations.
- Scapy
  - Packet manipulation and sniffing library.
  - Provides sniff() and packet objects, mainly used for capture on Linux and macOS.
  - Hides some OS-specific details (e.g. AF_PACKET vs libpcap), making the code more portable.
- Colorama
  - Adds color to terminal output in a cross platform way.

    - Used to highlight sections, warnings, and payload previews for better readability.
- Argparse
    - Implements a clean command line interface.
    - Makes the tool self documenting through -h help output.
- Pcap writer (dpkt or Scapy utils)
    - Writes captured packets to .pcap files.
    - Allows further analysis in Wireshark and other tools.

The combination of these libraries allows me to focus on the core logic and decoding while still retaining enough low level control to learn how packets are structured.

5. High Level Architecture

The sniffer is structured into several logical components:

1. Command Line Interface (CLI)
    - Parses user options.
    - Prints a startup summary showing interface, packet limit, save mode, filter, and verbosity.
2. Capture Backends
    - Use Scapy to capture packets on Linux and macOS.
    - Abstract away OS specific details like AF_PACKET and libpcap.
3. Packet Decoding Pipeline
    - Decodes Ethernet, IPv4, ICMP, TCP, and UDP headers from raw bytes.
    - Prints human readable summaries in the terminal.
4. Payload Handling
    - Displays payload length, previews, and hints by default.
    - Supports optional full hex dumps in verbose mode.
5. Capture Storage and Saving
    - Stores captured packets in memory with timestamps.
    - Saves them to .pcap files at the end of the session when configured.

Each captured packet flows through the pipeline: **capture → decode → print → store → optional save.**

6. Command Line Interface Design

The CLI is a core part of the user experience. It exposes the following arguments:

- -i, --interface
  Selects the network interface to sniff on (for example eth0, wlan0, en0).
- -c, --count
  Specifies the number of packets to capture.
    - 0 represents “unlimited” capture until the user interrupts with Ctrl+C.
- --filter
  A BPF filter string (e.g. "tcp port 80"), mainly used in the macOS/Scapy path.
- --save
  Controls when to save captured packets to a .pcap file:
  always – always save if there are packets.
  never – never save.
  ask – prompt the user at the end (y/n).
- --output-prefix
  Sets the prefix for the pcap filename, useful when organising multiple runs.
- --verbose-payload
  Enables full payload hex dumps in addition to short previews.

This design lets the tool be used in different ways:

- Quick tests: -c 10 --save never.
- Continuous monitoring with automatic save: --save always.
- Detailed debugging with verbose payload mode.

7. Capture Backends: Linux and macOS

The sniffer supports both Linux and macOS, which have different ways of capturing packets.

7.1 Linux Capture

On Linux, Scapy can use AF_PACKET sockets to access raw packets. The script:

- Detects that it is running on Linux and that AF_PACKET is available.
- Calls sniff() with:
    - the chosen interface,
    - an optional BPF filter, and
    - an optional count limit when count > 0.

Scapy delivers each captured packet to a callback function. In that callback, the script converts the packet to raw bytes and passes it into the shared decoding pipeline. This keeps the rest of the code independent of Scapy’s packet object representation.

7.2 macOS Capture

On macOS, Scapy uses libpcap as its underlying capture mechanism. The script:

- Detects that it is running on macOS (Darwin).
- Again uses sniff() with:
    - interface,
    - optional filter,
    - optional count.

macOS typically requires **sudo** for packet capture, which is reflected in the README and startup messages.

In both environments, a startup summary is printed so that the user can confirm:

1. Which interface is being used.
2. Whether the packet limit is finite or unlimited.
3. Which save policy is active.
4. Whether a filter or verbose mode is enabled.

8. Packet Decoding Pipeline

The decoding pipeline handles each packet in a sequence of layers. This mirrors the OSI/network stack and reinforces how each layer encapsulates the next.

8.1 Ethernet Layer

The first 14 bytes of the raw frame are interpreted as:

- Destination MAC address (6 bytes)
- Source MAC address (6 bytes)
- EtherType (2 bytes)

The EtherType field indicates which protocol is encapsulated next. For example:

- 0x0800 – IPv4
- Other values – not handled in depth for this project, but still displayed numerically.

MAC addresses are displayed in the standard AA:BB:CC:DD:EE:FF format, which helps visually identify local vs remote addresses if you know your hardware addresses.

8.2 IPv4 Layer

If the EtherType is IPv4, the script decodes:

- Version (should be 4 for IPv4).
- Header length (ignoring options in this simple implementation).
- Time to Live (TTL).
- Protocol number:
    - 1 = ICMP
    - 6 = TCP
    - 17 = UDP
- Source IP address.
- Destination IP address.

The addresses are printed in dotted decimal notation (e.g. 192.168.1.10). The header length is used to locate where the IP header ends and where the transport layer begins.

8.3 ICMP

When the protocol is 1, the script decodes:

- Type (e.g. 8 for echo request, 0 for echo reply).
- Code (additional detail about the message).
- Checksum.

Any remaining bytes are considered ICMP payload and passed to the payload display routine. This is useful when, for example, watching ping requests and replies.

8.4 TCP

When the protocol is 6 (TCP), the script decodes:

- Source port and destination port.
- Sequence number and acknowledgment number.
- Header offset and flags, including:
    - URG, ACK, PSH, RST, SYN, FIN.

These fields are crucial for understanding TCP connection setup (three way handshake), teardown, and data flow. The remaining bytes are treated as payload and displayed accordingly.

8.5 UDP

When the protocol is 17 (UDP), the script decodes:

- Source port and destination port.

- Length of the UDP segment.

The rest is payload. UDP is simpler than TCP because it is connectionless and has fewer header fields.

For other protocol numbers, the script prints a message indicating that the transport protocol is unsupported in this educational version. This keeps the scope focused while still clearly informing the user.

9. Payload Display and Heuristic Hints

Dumping full payload data for every packet would make the output unreadable, especially on busy networks. To avoid that, the project uses a two level approach.

9.1 Default Behavior: Preview plus Hint

By default, the script prints:

- Payload length in bytes.
- Hex preview of the first few bytes (for example, the first 32 bytes).
- A simple hint when a recognizable pattern appears.

Examples of hints:

- “Possible IPv6 payload” when the first nibble suggests version 6.
- “Possible IPv4 payload” for version 4.
- “Possible TLS application data” when the prefix matches a typical TLS record header.

These hints are intentionally conservative. They are meant to provide quick context, not full protocol identification.

9.2 Verbose Mode: Full Hex Dump

When the --verbose-payload flag is set, the script also prints a full hex dump of the payload. The bytes are encoded as \xNN sequences, wrapped across multiple lines to fit the terminal width.

This mode is useful in more advanced situations:

- Inspecting custom or unusual protocols.
- Checking whether payloads contain particular byte patterns.
- Debugging encryption wrappers or padding.

10. Colored Terminal Output

Human readability is a key goal. To help the eye quickly parse output during live capture, the tool uses colored terminal output through the Colorama library.

The color scheme is:

- Cyan – labels and section headings (e.g. “Ethernet Frame”, “IPv4 Packet”).
- Green – successfully decoded transport sections (e.g. “TCP Segment”, “UDP Segment”).
- Yellow – warnings, hints, and non fatal issues.
- Red – errors such as decode failures or permission problems.
- Magenta – packet titles and payload section headings.
- Dim gray – payload previews, to make them visually distinct but not dominant.

Colorama is treated as an optional dependency:

- If installed, colors are enabled.
- If missing, the script falls back to plain text without raising an error.

This approach improves the usability of the tool without making it fragile.

11. Capture Storage and Saving to .pcap

For each captured packet, the script:

- Increments a global packet counter.
- Stores a tuple (timestamp, raw_bytes) in a list.
- Prints a structured view in the terminal.

At the end of the session, the script:

- Prints the total number of packets captured.
- Decides whether to save based on the --save option:
    - If always, it saves automatically.
    - If never, it skips saving.
    - If ask, it prompts the user.

If saving is chosen and there are captured packets, a .pcap file is written with a name like:

capture_20260428_103018.pcap

This file can be opened in Wireshark to inspect packets with more detail, apply filters, and explore protocol fields interactively.

The save logic runs in a final cleanup path, so it still executes when the user stops capture with Ctrl+C. This makes continuous capture with automatic save reliable.

12. Example Use Cases

The sniffer can be used in multiple practical scenarios:

- Observing ping traffic
  - Start the sniffer and run ping 8.8.8.8.
  - Watch ICMP echo requests and replies, including types and codes.
- Watching HTTP/HTTPS traffic
- Run the sniffer with a filter for port 80 or 443.
- Open a website and observe TCP handshakes and TLS records.

- Monitoring DNS queries
  - Filter for UDP port 53.
  - Look at DNS requests and responses.
- Capturing for later analysis
  - Use --save always to create pcap files.
  - Load them in Wireshark for more advanced inspection and protocol analysis.

These scenarios make the project not only educational but also useful as a lightweight capture tool.

13. Challenges and Problem Solving

Several challenges emerged while building this project:

1. Platform differences
   a. Linux and macOS handle raw packet capture differently.
   b. Scapy helped abstract this, but I still had to detect OS and choose the right path.
2. Callback exceptions stopping capture
   a. A bug in the packet processing callback can cause Scapy's socket to close after the first packet.
   b. I had to carefully align function signatures and handle exceptions gracefully to avoid silent failures.

3. Handling Ctrl+C cleanly
    a. I needed to ensure that interrupting the program would:
        i. stop capture,
        ii. print a summary, and
        iii. still offer to save captured packets.

    b. This required proper use of try/except and finally blocks.
4. Balancing detail vs readability
    a. Full hex dumps are powerful but overwhelming.
    b. Designing a preview plus hint system, with optional verbose mode, was a key UX decision.

These challenges reinforced good habits such as defensive programming, careful error handling, and iterative testing.

14. Skills and Learning Outcomes

From this project, I developed and demonstrated several skills:

- Networking fundamentals
    - Understanding of Ethernet, IPv4, ICMP, TCP, and UDP at the byte level.
    - Ability to connect these details to practical tools like Wireshark.
- Python for systems and security
    - Experience with Scapy, raw packet structures, and CLI tools.
    - Use of third party libraries in a robust, optional way (e.g. Colorama fallback).
- Debugging and problem solving
    - Tracing issues with callbacks, signals, permissions, and cross platform behavior.
    - Systematically fixing and testing edge cases.
- Usability and developer experience
    - Designing clear terminal output with color and structure.
    - Providing meaningful CLI options and help text.
    - Writing a README and technical report to make the tool understandable to others.

These outcomes are directly relevant to security engineering roles, SOC analysis, and network troubleshooting.

15. Future Work

Potential future enhancements include:

- IPv6 decoding
  - Extending the pipeline to understand IPv6 headers.
- Application layer hints
  - Basic recognition of HTTP, DNS, and other application protocols.
- Streaming capture to disk
  - Writing packets directly to disk instead of storing everything in memory.
- Profiles and presets
  - Providing command line presets for common tasks (e.g. “web”, “dns”, “ping”).

Each of these would further deepen the project and align it even more with real world security workflows.

16. Conclusion

This project started from a simple question: What really happens on a network when data is sent and received? By building a custom packet sniffer in Python, I was able to answer that question in a concrete way.

The resulting tool captures real traffic on Linux and macOS, decodes key protocols, shows payload previews and hints, and saves to .pcap for analysis in Wireshark. More importantly, the process of designing, implementing, debugging, and documenting the sniffer has strengthened my understanding of networks, my Python skills, and my ability to build security focused tools that are practical and user friendly.